Comprehensive Analysis: Why DeFRCN Modifications Fail to Surpass Vanilla Results and Proposed Novel Solutions for Top-Tier Venues

FSOD Research Analysis

March 2026

Abstract

This document provides a comprehensive analysis of experimental results obtained from running DeFRCN (Decoupled Faster R-CNN) with various modifications across three remote machines (bio, ndag, ssl) for few-shot object detection on PASCAL VOC. Despite implementing multiple modifications—including adapter ablations, modified PCB (Prototypical Calibration Block) variants, transductive inference, novel calibration methods (contrastive anchoring, self-distillation, frequency augmentation), and Res5 adapter ablations—the results consistently fail to surpass the paper-reported numbers and often do not meaningfully improve over the vanilla DeFRCN baseline. We identify **seven root causes** for this performance gap and propose **five novel research directions** with sufficient novelty and rigor for publication at CVPR, ICLR, or A*-level conferences.

Contents

1 Experimental Setup Summary **3**
1.1 Paper's Original Configuration . . . 3
1.2 Our Configuration . . . 3
1.3 Hardware Deployed . . . 3

2 Comprehensive Results Comparison **3**
2.1 DeFRCN Paper Reported Results (VOC Split 1, FSOD, AP_{50}) . . . 3
2.2 Our Results: All Modifications (nAP_{50}, VOC Split 1) . . . 3
2.3 Critical Observation: Identical Novel Method Results . . . 3

3 Root Cause Analysis: Seven Reasons for Failure **5**
3.1 Reason 1: Batch Size Reduction (16 → 8) — The Dominant Factor . . . 5
3.2 Reason 2: Single Run vs. 10-Repeat Averaging (CORRECTED) . . . 5
3.3 Reason 3: Novel Methods Not Actually Executing . . . 6
3.4 Reason 4: PCB Modifications Operate in a Narrow Band . . . 6
3.5 Reason 5: Adapter Modules Overfit in Few-Shot Regime . . . 6
3.6 Reason 6: Base Model Quality Ceiling . . . 7
3.7 Reason 7: Hyperparameter Transfer Assumption . . . 7

4 Summary of Gap Attribution **7**

5 Proposed Novel Solutions for A*-Level Publication **7**
5.1 Solution 1: Batch-Agnostic Few-Shot Detection via Normalization Reparameterization (BA-FSD) . . . 7
5.2 Solution 2: Dual-Pathway Gradient Decoupling with Learnable Task Routing (DP-GDL) . . . 8
5.3 Solution 3: Prototypical Calibration with Foundation Model Alignment (PCB-FMA) . . . 9
5.4 Solution 4: Meta-Calibration: Learning to Calibrate Across Episodes (Meta-PCB) 9
5.5 Solution 5: Uncertainty-Guided Prototype Refinement with Test-Time Adaptation (UPR-TTA) . . . 10

6 Recommended Immediate Actions **10**

7 Publication Strategy **11**

8 Conclusion **11**

1 Experimental Setup Summary

1.1 Paper's Original Configuration

The DeFRCN paper reports the following implementation details:

- **Batch size**: 16 (mini-batch)
- **Base LR**: 0.02 (base training), 0.01 (novel fine-tuning)
- **Optimizer**: SGD, momentum 0.9, weight decay 5×10^{-5}
- **Backbone**: ResNet-101 pre-trained on ImageNet
- **Evaluation**: Results averaged over *multiple random runs*
- **GPU**: 8 GPUs (`--num-gpus 8` in official script), 2 images/GPU

1.2 Our Configuration

- **Batch size**: 8 (halved from 16 due to GPU memory constraints)
- **Base LR**: 0.01 (linearly scaled from 0.02 for BS=8)
- **Novel LR**: 0.005 (linearly scaled from 0.01 for BS=8)
- **Base training**: 30,000 iterations (doubled from paper's 15K to match throughput), steps at (20000, 26600), warmup 100
- **Novel fine-tuning**: 1,600 iterations (doubled from paper's 800), steps at (1280,), no warmup
- **Repeats**: 1 (paper runs 10 repeats and averages)
- **GPU**: Single NVIDIA RTX 3090 (24GB)
- **Evaluation**: Single seed (seed=0), single split (split 1)

1.3 Hardware Deployed

Table 1: Remote machines used for experiments

Host	Repo Path	Results Files	GPU
bio	/home/bio/Tanvir_Saikat/fsod-test	14	RTX 3090
ndag	/home/ndag/Documents/fsod-test	4	—
ssl	/home/cse/Documents/fsod-test	16	—

2 Comprehensive Results Comparison

2.1 DeFRCN Paper Reported Results (VOC Split 1, FSOD, AP_{50})

Table 2: Paper-reported DeFRCN results on PASCAL VOC Novel Set 1 (FSOD, AP_{50})

	1-shot	2-shot	3-shot	5-shot	10-shot
DeFRCN (Paper)	**53.6**	**57.5**	**61.5**	**64.1**	**60.8**

2.2 Our Results: All Modifications (nAP_{50}, VOC Split 1)

2.3 Critical Observation: Identical Novel Method Results

A deeply concerning finding: **contrastive anchoring, self-distillation, and frequency augmentation produce *bit-for-bit identical* results** for all shots ≥ 2:

Table 3: Complete results inventory across all three machines. Δ shows difference from paper's 1-shot AP50 of 53.6 where applicable.

Method	**1-shot**	**10-shot**	**Δ (1s)**	**Host**
Vanilla / Baseline				
Proto Init Baseline	50.12	—	-3.48	bio
PCB Modifications (10-shot, eval-only on vanilla ckpts)				
Quality Weighted	—	60.13	—	ssl
Scale Aware	—	60.21	—	ssl
Multi Prototype	—	59.78	—	ssl
Class Conditional Gate	—	58.16	—	ssl
Score Normalization	—	57.78	—	ssl
Adaptive Alpha	—	55.58	—	ssl
Robust Aggregation	—	55.74	—	ssl
Transductive PCB Combinations				
Transductive (bio)	51.80	57.73	-1.80	bio
Trans.+Scale Aware (iter2)	50.13	—	-3.47	ssl
Trans.+Scale Aware (iter3)	50.48	—	-3.12	ssl
Trans.+Scale Aware (new)	50.62	—	-2.98	ssl
Trans.+Quality Weighted	51.22	58.18	-2.38	ssl
Trans.+Multi Prototype	51.52	58.78	-2.08	ssl
Novel Methods (eval-only on vanilla ckpts)				
Contrastive Anchoring	49.54	56.56	-4.06	bio
Self Distillation	49.54	56.56	-4.06	bio
Frequency Augmentation	—	56.56	—	bio
Adapter Ablations (RPN/ROI-level adapters, fine-tuned)				
Adapter Baseline (bio, no split1)	46.49	—	-7.11	bio
Adapter Baseline (ssl, split1)	50.78	—	-2.82	ssl
LN Adapter	50.34	—	-3.26	bio
RPN Only Adapter	50.03	—	-3.57	bio
ROI Only Adapter	50.23	—	-3.37	bio
Shared Adapter	49.52	—	-4.08	ndag
Gate Init 1	49.33	56.73	-4.27	ndag
Heavy Adapter	48.63	—	-4.97	ndag
Light Adapter	49.89	—	-3.71	ndag
Res5 Adapter Ablations				
All Blocks (bio)	25.51	—	-28.1	bio
All Blocks (ssl)	47.13	—	-6.47	ssl
Heavy (bio)	19.78	—	-33.8	bio
Heavy (ssl)	48.32	—	-5.28	ssl
Block 1_2 (bio)	24.65	—	-29.0	bio
Block 1_2 (ssl)	47.36	—	-6.24	ssl
RPN Frozen	19.60	—	-34.0	bio
Res5 Baseline 10-shot	—	53.57	—	bio

This proves that the novel methods **are not actually modifying the inference pipeline**. They load vanilla model weights via `infer_pretrained_novel` mode and the novel method code paths either: (a) have no effect on the PCB calibration scores, (b) are not triggered due to configuration, or (c) modify features that get overwritten by the standard PCB pipeline. These are effectively just vanilla DeFRCN results.

Table 4: Identical results across "different" novel methods (nAP_{50})

Method	2-shot	3-shot	5-shot	10-shot
Contrastive Anchoring	56.07	54.93	60.52	56.56
Self Distillation	56.07	54.93	60.52	56.56
Frequency Augmentation	56.07	54.93	60.52	56.56

3 Root Cause Analysis: Seven Reasons for Failure

3.1 Reason 1: Batch Size Reduction (16 → 8) — The Dominant Factor

The DeFRCN paper uses a mini-batch size of 16. Due to GPU memory constraints (single RTX 3090, 24GB), we halved this to 8 and applied *linear scaling* to the learning rate (0.02 → 0.01 for base, 0.01 → 0.005 for novel).

Why linear scaling is insufficient:

1. **Gradient noise amplification:** With BS=8, each gradient estimate has $\sqrt{16/8} = \sqrt{2} \approx 1.41\times$ higher variance compared to BS=16. In the few-shot regime where data is already scarce, this compounds severely.
2. **BatchNorm statistics degradation:** ResNet-101 uses Batch Normalization. With BS=8, the running mean and variance estimates during *base training* are computed from only 8 samples per iteration instead of 16. This produces noisier batch statistics that get frozen into the model. Since DeFRCN freezes most parameters during novel fine-tuning, these suboptimal BN statistics propagate directly to novel-class evaluation.
3. **Non-linear LR scaling regime:** The linear scaling rule (Goyal et al., 2017) was validated for large-batch training (BS $\geq$ 256). For small batches (BS $\leq$ 16), the relationship between batch size and optimal LR is not strictly linear. Recent work (Hoffer et al., 2017; McCandlish et al., 2018) shows that the optimal LR for small batches can be *sub-linear* or require warmup schedules that differ from simple scaling.
4. **Effective number of gradient updates:** With the same number of iterations (30K for base), BS=8 sees $30000 \times 8 = 240$K images while BS=16 sees $30000 \times 16 = 480$K images. **We have only seen half the training data**. Alternatively, if we matched the total images seen, we would need 60K iterations—double the current schedule.

Estimated impact: 2–4 AP_{50} points on novel classes. This alone accounts for most of the gap between our 50.12 and the paper's 53.6.

3.2 Reason 2: Single Run vs. 10-Repeat Averaging (CORRECTED)

Correction: The official DeFRCN `run_voc.sh` reveals that for VOC FSOD, the paper uses the "FSRW-like" protocol: **seed 0 only, repeated 10 times**. The comment in the script states: *"run seed0 10 times (the FSOD results on voc in most papers)"*. Therefore, both the paper and our experiments use seed 0—seed selection is **NOT** a factor in the performance gap.

However, the paper averages 10 independent training runs (same support images, different random initialization and mini-batch order). We run only once. This reduces the paper's variance from stochastic training.

Why this still matters (but less than originally claimed):

- 10 repeats of the same seed averages out *training stochasticity* (random init of novel classifier head, mini-batch sampling order, dropout).
- Our single run could be unlucky (above or below the 10-run mean).
- However, the support images are identical—the dominant source of FSOD variance (which specific K images are chosen) is eliminated.

Estimated impact: ±1–2 AP_{50} points (much less than the ±2–5 for different seeds).

3.3 Reason 3: Novel Methods Not Actually Executing

As shown in Table 4, three supposedly distinct methods produce *identical* results. Investigation reveals:

1. The `infer_pretrained_novel` run mode loads **vanilla DeFRCN weights** and only applies novel method modifications at inference time.
2. The novel methods (contrastive anchoring, self-distillation, frequency augmentation) are implemented as wrappers around the PCB module in `defrcn/evaluation/novel_methods/`.
3. However, these methods likely either:
   - Fall through to the base `execute_calibration()` without modification
   - Apply transformations that are numerically negligible (e.g., zero-weighted terms)
   - Only activate during training but are bypassed during `--eval-only`
4. The `build_novel_method_pcb()` wrapper in `evaluator.py:97` wraps the PCB, but if the wrapped method's `execute_calibration()` simply delegates to the base PCB, no change occurs.

Conclusion: These results are **not** evidence that the methods fail; they are evidence that the methods **are not being applied**. This is a code integration issue, not a methodology issue.

3.4 Reason 4: PCB Modifications Operate in a Narrow Band

The modified PCB variants (quality-weighted, multi-prototype, scale-aware, transductive, etc.) only affect the *score calibration step*—the final $s_i^\dagger = \alpha \cdot s_i + (1-\alpha) \cdot s_i^{cos}$ computation. This has inherent limitations:

1. **PCB is a post-hoc correction:** It cannot recover from poor proposals or misaligned features. If the Faster R-CNN backbone produces bad RoI features, no amount of prototype calibration fixes this.
2. $\alpha = 0.5$ **saturation:** The default $\alpha = 0.5$ gives equal weight to detector scores and prototype similarity. Most PCB modifications only slightly shift this balance. The gap between $\alpha = 0.5$ and the optimal α is typically < 1 AP_{50}.
3. **ImageNet feature extractor bottleneck:** PCB uses an *independently pre-trained* ResNet-101 on ImageNet as the feature extractor for prototypes. This creates a **domain mismatch**—ImageNet features are optimized for 1000-class classification, not for few-shot VOC object detection. The cosine similarity scores are bounded by this mismatch.
4. **Transductive inference ceiling:** Transductive methods collect pseudo-labels from high-confidence detections, but in 1-shot, the detector's initial recall is so low that very few pseudo-labels are collected. The ssl log shows only 25 pseudo samples across 5 classes from 4952 test images—5 per class. This barely augments the original 1 shot per class.

Estimated impact: PCB modifications can contribute at most 1–2 AP_{50} points, which aligns with observed results (51.80 vs 50.12 for transductive on bio).

3.5 Reason 5: Adapter Modules Overfit in Few-Shot Regime

The adapter ablations (RPN-only, ROI-only, shared, heavy, light, gate, LN, Res5 adapters) consistently perform *at or below* vanilla DeFRCN:

- **Best adapter (LN):** 50.34 vs vanilla 50.12 (+0.22, negligible)
- **Worst adapter (Heavy, bio Res5):** 19.78 (−30.34, catastrophic)
- **Average adapter:** ∼49.5 (worse than vanilla)

Root causes for adapter failure:

1. **Insufficient data for adapter parameters:** In 1-shot, we have only 5 training images for 5 novel classes. Adapters add hundreds to thousands of new parameters. Even "lightweight" adapters with bottleneck dimension 64 add $2 \times C \times 64$ parameters per layer, which is far too many relative to the 5 training images.

2. **Redundancy with GDL:** DeFRCN's Gradient Decoupled Layer already handles the multi-stage decoupling that adapters are meant to provide. Adding adapters on top of GDL creates redundant capacity that degrades generalization.
3. **Res5 adapter catastrophic failure (bio):** The drastically different results between bio (19.78–25.51) and ssl (47.13–48.32) for the same Res5 adapter configurations indicate either (a) different code versions, (b) different base weights, or (c) a training bug on bio. The bio machine shows evidence of uncommitted local changes that conflicted with merges.
4. **Learning rate mismatch:** Adapter modules require careful LR tuning separate from the backbone. Using the same LR for adapters as for the classifier head leads to sub-optimal convergence.

3.6 Reason 6: Base Model Quality Ceiling

The base model (trained on 15 base classes with BS=8) establishes a performance ceiling for all downstream few-shot experiments. The base training log reveals:

- BS=8 with LR=0.01, 30K iterations, steps at (20000, 26600)
- The paper uses BS=16 with LR=0.02, implying the same iteration schedule but **2× the data throughput**
- A weaker base model produces worse RPN proposals and worse Res5 features for novel classes
- This propagates through model surgery (weight resetting) into the novel fine-tuning stage

Estimated impact: 1–3 AP_{50} points. The base model quality directly affects the backbone features available for novel class adaptation.

3.7 Reason 7: Hyperparameter Transfer Assumption

All modifications assume the DeFRCN hyperparameters (GDL λ values, PCB α, score thresholds, LR schedules) remain optimal when batch size changes. This is fundamentally flawed:

- The optimal $\lambda_{rpn} = 0$ and $\lambda_{rcnn} = 0.75$ were tuned with BS=16. With BS=8, the gradient magnitudes change, altering the optimal decoupling balance.
- The PCB $\alpha = 0.5$ was calibrated for models trained with BS=16. The detector confidence distribution shifts with BS=8 training.
- Novel fine-tuning schedule (1600 iter for 1-shot) was designed for BS=16 ($1600 \times 16 = 25600$ sample-steps). With BS=8, we only get $1600 \times 8 = 12800$ sample-steps—half the effective training.

4 Summary of Gap Attribution

5 Proposed Novel Solutions for A*-Level Publication

Below we propose five research directions, each with sufficient novelty, technical depth, and experimental validation potential for CVPR, ICLR, NeurIPS, or equivalent A*-level venues.

5.1 Solution 1: Batch-Agnostic Few-Shot Detection via Normalization Reparameterization (BA-FSD)

Core Idea: Replace BatchNorm with a batch-size-invariant normalization scheme *specifically designed for the two-stage few-shot detection pipeline*, enabling consistent performance regardless of batch size.

Technical Approach:

Table 5: Estimated contribution of each factor to the $\sim$3.5 AP_{50} gap (1-shot, Split 1)

Factor	**Est. Impact (AP_{50})**	**Fixable?**
Batch size 16 $\rightarrow$ 8 (base training)	-1.5 to -3.0	Partially
Batch size 16 $\rightarrow$ 8 (novel fine-tuning)	-0.5 to -1.5	Partially
Single run vs. 10-repeat average	±1.0 to ±2.0	Yes (run 10$\times$)
Hyperparameter transfer (non-retuned)	-0.5 to -1.5	Yes
Novel methods not executing	N/A (bug)	Yes
Half effective training data	-0.5 to -1.0	Yes (2$\times$ iters)
Total estimated gap	~-3 to -7	—
Observed gap	-3.48	—

1. Replace all BN layers in ResNet-101 with **Group Normalization (GN) + Weight Standardization (WS)** during *base training.* This eliminates the dependency on batch statistics entirely.
2. During novel fine-tuning, apply **Task-Adaptive Normalization (TAN)**: learn lightweight affine parameters $(\gamma_{novel}, \beta_{novel})$ for each normalization layer conditioned on the support set statistics, using a hypernetwork that takes the K-shot support features as input.
3. Key insight: BN's failure in small-batch FSOD is not just about noisy statistics—it's about the *distributional shift* between base classes (many samples) and novel classes (few samples). TAN explicitly models this shift.

Novelty: While GN has been used in detection (Wu & He, 2018), no prior work addresses the *compound effect* of small-batch BN on the base$\rightarrow$novel transfer pipeline in FSOD, nor proposes conditional normalization adaptation for few-shot detection.

Expected Contribution:

- Batch-size-invariant FSOD performance (BS=2 to BS=64 with < 1 AP variance)
- +2–4 AP improvement for small-batch settings
- Drop-in replacement applicable to any Faster R-CNN based FSOD method

5.2 Solution 2: Dual-Pathway Gradient Decoupling with Learnable Task Routing (DP-GDL)

Core Idea: Extend DeFRCN's Gradient Decoupled Layer from a *static* decoupling coefficient to a *dynamic, input-dependent* routing mechanism that learns which gradient pathways to activate based on the task context (base vs. novel, classification vs. localization).

Technical Approach:

1. Replace the fixed scalar λ in GDL with a **learnable gating network** $G_\phi(x)$ that outputs per-channel routing weights: $\lambda_c = \sigma(G_\phi(\text{GAP}(x)))$, where GAP is global average pooling.
2. Introduce a **dual pathway**: one branch optimizes for translation-invariant features (classification) and the other for translation-covariant features (localization). The gating network learns to route features appropriately.
3. During novel fine-tuning, freeze the gating network's base-trained weights but add a **residual task-adaptation vector** $\Delta\phi_{novel}$ that is learned from the K-shot support set.
4. Loss: $\mathcal{L} = \mathcal{L}_{det} + \lambda_{orth}\|\nabla_{cls} \cdot \nabla_{loc}\|^2$, where the orthogonality regularizer encourages the two pathways to learn complementary features.

Novelty: DeFRCN's GDL uses fixed, manually-tuned λ values. This fundamentally limits adaptability. No prior work in FSOD learns input-dependent gradient routing with explicit task decomposition.

Expected Contribution:

- Eliminates manual λ tuning (currently requires grid search per dataset/shot)

- +1–3 AP from better gradient flow management
- Theoretically grounded: connects to mixture-of-experts and conditional computation literature

5.3 Solution 3: Prototypical Calibration with Foundation Model Alignment (PCB-FMA)

Core Idea: Replace the ImageNet-pretrained ResNet-101 feature extractor in PCB with a **vision-language foundation model** (e.g., DINOv2, SigLIP) and perform calibration in the foundation model's feature space with learned projection alignment.

Technical Approach:

1. Extract RoI features from both the detector's backbone *and* a frozen foundation model (e.g., DINOv2-g).
2. Learn a lightweight **cross-space projection** $\pi : \mathbb{R}^{d_{det}} \to \mathbb{R}^{d_{fm}}$ that aligns detector features to the foundation model's semantic space. Train π on base classes with a contrastive objective.
3. Compute PCB prototypes in the *foundation model's* feature space, which provides:
   - Much richer semantic representations (trained on billions of images)
   - Better generalization to novel classes (foundation models are designed for transfer)
   - Optional text-conditioned prototypes using class names ("bird", "bus", "cow", etc.)
4. Fusion: $s_i^\dagger = \alpha \cdot s_i^{det} + \beta \cdot s_i^{fm} + \gamma \cdot s_i^{text}$, where α, β, γ are learned via the K-shot support set.

Novelty: No prior FSOD work integrates foundation model features into the PCB calibration pipeline. The cross-space projection and tri-modal score fusion (detector + visual foundation + text) is novel.

Expected Contribution:

- +3–8 AP from foundation model's superior generalization
- Zero-shot capability via text-conditioned prototypes
- Maintains DeFRCN's efficiency (foundation model is frozen, projection is tiny)
- Bridges the FSOD and foundation model communities

5.4 Solution 4: Meta-Calibration: Learning to Calibrate Across Episodes (Meta-PCB)

Core Idea: Instead of computing prototypes in a fixed, hand-designed manner, *learn the calibration function itself* through meta-learning over simulated few-shot episodes constructed from base classes.

Technical Approach:

1. During base training, construct simulated FSOD episodes: randomly select 5 classes as "pseudo-novel," withhold all but K examples, and train the model.
2. Learn a **calibration network** $\mathcal{C}_\theta$ that takes as input: (a) detector score s_i, (b) RoI feature x_i, (c) support set features $\{x_j^{support}\}$, and outputs the calibrated score.
3. The calibration network replaces the fixed cosine-similarity + alpha-blending formula with a **learned, non-linear calibration function** that can capture complex score-feature interactions.
4. Meta-objective: $\min_\theta \mathbb{E}_{episode}\left[\mathcal{L}_{det}(\mathcal{C}_\theta(s, x, S))\right]$, trained with MAML-style bi-level optimization or Prototypical Networks-style episodic training.
5. At novel evaluation time, $\mathcal{C}_\theta$ is applied zero-shot (no fine-tuning needed for the calibration module itself).

Novelty: PCB's calibration formula ($s_i^\dagger = \alpha \cdot s_i + (1-\alpha) \cdot s_i^{cos}$) is a *fixed, linear* function. Learning a non-linear calibration function via meta-learning is entirely novel in the FSOD literature.

Expected Contribution:

- Replaces 7+ hand-tuned PCB hyperparameters (α, upper/lower thresholds, quality weights, etc.) with a single learned module
- +2–5 AP from non-linear calibration
- Meta-learning ensures generalization to truly unseen novel classes
- Plug-and-play: can be applied to any detection framework

5.5 Solution 5: Uncertainty-Guided Prototype Refinement with Test-Time Adaptation (UPR-TTA)

Core Idea: Combine Bayesian uncertainty estimation with test-time adaptation to *continuously refine* prototypes during inference, using prediction confidence as a self-supervisory signal.

Technical Approach:

1. **Epistemic uncertainty via MC Dropout:** During inference, apply dropout in the Res5 head (which is normally frozen) and perform T stochastic forward passes. The variance of predictions across passes estimates epistemic uncertainty.
2. **Uncertainty-guided prototype refinement:**
   - For low-uncertainty, high-confidence detections: incorporate as pseudo-support to refine prototypes (similar to transductive, but principled).
   - For high-uncertainty detections: increase the PCB's influence (α decreases) to rely more on prototype matching.
   - For moderate uncertainty: use the uncertainty estimate to *weight* the prototype contribution dynamically.
3. **Test-time adaptation (TTA):** Use the pseudo-labeled high-confidence detections to perform *one-step gradient updates* on the classifier head during inference (inspired by TENT, but adapted for detection).
4. Formalize via a Bayesian framework: $p(y|x, S) = \int p(y|x, \theta)p(\theta|S)d\theta$, where S is the support set and the posterior $p(\theta|S)$ is approximated via the refined prototypes.

Novelty: No prior FSOD work combines: (a) epistemic uncertainty estimation in the detection head, (b) uncertainty-guided PCB modulation, and (c) test-time adaptation. The Bayesian formulation provides theoretical grounding that existing transductive methods lack.

Expected Contribution:

- +2–4 AP from principled prototype refinement (vs. ad-hoc transductive thresholds)
- Robustness: uncertainty prevents incorporation of false positives as pseudo-support
- TTA provides domain adaptation benefits for cross-domain FSOD
- Clean Bayesian formulation suitable for ICLR/NeurIPS

6 Recommended Immediate Actions

Before pursuing novel contributions, we recommend fixing the experimental setup:

1. **Fix novel methods code**: Debug why contrastive anchoring, self-distillation, and frequency augmentation produce identical results. The methods are not executing.

2. **Double base training iterations**: Increase from 30K to 60K iterations to compensate for BS=8 seeing half the data. Alternatively, use gradient accumulation to simulate BS=16.

3. **Run multiple seeds**: Evaluate seeds 0–9 and report mean ± std. This is essential for valid comparison with the paper.

4. **Re-tune hyperparameters for BS=8**: Grid search over:

- Novel LR: {0.003, 0.005, 0.008, 0.01}
- PCB α: {0.3, 0.4, 0.5, 0.6, 0.7}
- Novel iterations: {1600, 2400, 3200} (1-shot)
- GDL λ_{rcnn}: {0.5, 0.75, 1.0}

5. **Reconcile bio vs. ssl Res5 adapter results**: Investigate why the same configuration produces AP50=19.78 on bio but 48.32 on ssl. Check git state, code versions, and base weights.

6. **Consider gradient accumulation**: Accumulate gradients over 2 forward passes before updating, effectively simulating BS=16 within the memory constraints of a single RTX 3090.

7 Publication Strategy

Table 6: Recommended target venues for each proposed solution

Solution	**Target Venue**	**Rationale**
BA-FSD (Batch-Agnostic)	CVPR / ECCV	Practical impact, extensive experiments
DP-GDL (Dual-Pathway)	CVPR / ICCV	Architectural novelty, ablation-rich
PCB-FMA (Foundation Model)	ICLR / NeurIPS	Timely (foundation models), strong gains
Meta-PCB (Meta-Calibration)	NeurIPS / ICML	Theoretical depth, meta-learning
UPR-TTA (Uncertainty)	ICLR / AAAI	Bayesian framework, principled

Strongest recommendation: Solution 3 (PCB-FMA) is the most publishable due to (a) the timeliness of foundation models, (b) the expected large performance gain, (c) simplicity of implementation, and (d) broad applicability. Combined with Solution 4 (Meta-PCB) as an ablation or extension, this forms a compelling CVPR/ICLR paper.

Paper title suggestion: "*Beyond Cosine Similarity: Foundation Model-Aligned Prototypical Calibration for Few-Shot Object Detection*"

8 Conclusion

The performance gap between our DeFRCN reproduction and the paper-reported results stems primarily from (1) batch size reduction without sufficient compensation, (2) single-seed evaluation, and (3) novel methods not being properly integrated. The modifications we attempted (PCB enhancements, adapters, transductive inference) operate in a fundamentally limited space—they only adjust *post-hoc* scores or add parameters that overfit with K-shot data.

For a genuinely impactful contribution, we must address the problem at a deeper level: either by fixing the normalization pipeline for batch-agnostic training, replacing the outdated ImageNet feature extractor with foundation models, or learning the calibration function itself through meta-learning. These directions offer both strong empirical gains and sufficient novelty for top-tier publication.